

# C Basic Syntax and Program Layout

Focused revision sheet for the RAG item: main, includes, blocks, semicolons, printf, escape sequences, variables, stack allocation, static typing, and format specifiers.

## Description: What You Need To Know

### 1. Minimal C program shape

A normal C program starts running in `main`. Header files are included before compilation so the compiler knows about library functions such as `printf`.

```
#include <stdio.h>

int main(void) {
    printf("Hello\n");
    return 0;
}
```

- `#include <stdio.h>` gives declarations for standard input and output functions.
- `int main(void)` says the program takes no command-line arguments and returns an integer status.
- `return 0;` means successful program completion.

### 2. Statements, semicolons, and blocks

Most C statements end with a semicolon. Curly braces group statements into a block, usually for a function, loop, or branch.

```
int x = 4;           /* declaration statement */
x = x + 1;           /* assignment statement */
printf("%d\n", x);   /* function-call statement */

if (x > 0) {
    printf("positive\n");
}
```

A missing semicolon usually causes the compiler to complain on the next line because it is still trying to parse the unfinished statement.

### 3. Comments

Comments are ignored by the compiler. Use them to explain intent, not to repeat obvious code.

```
// One-line comment

/*
Multi-line comment
*/
```

### 4. Static typing and automatic variables

C is statically typed: each variable has a declared type, and that type controls how the stored bits are interpreted.

Local variables declared inside a block are automatic variables. They normally live until execution leaves that block.

```
int main(void) {
    int count = 3;
    double price = 2.50;
    char grade = 'A';

    count = count + 1;
    /* count = "hello"; not valid: count is an int */

    return 0;
}
```

Automatic local variables are commonly stored on the call stack. Do not return a pointer to a local variable, because the variable stops existing after the function returns.

### 5. Printing with printf

`printf` uses a format string. Each conversion specifier must match the type of the argument you pass.

```
int count = 4;
double average = 2.75;
char letter = 'Z';
char name[] = "Ada";

printf("count=%d\n", count);
```

```
printf("average=%f\n", average);
printf("letter=%c\n", letter);
printf("name=%s\n", name);
```

- %d prints an int.
- %f prints a double.
- %c prints a single character.
- %s prints a null-terminated C string.
- %zu prints a size\_t value.

## 6. Escape sequences

Escape sequences let you put special characters into strings and character constants.

```
printf("first line\nsecond line\n");
printf("column1\tcolumn2\n");
printf("She said \"hi\"\n");
printf("backslash: \\ \n");
```

The source code contains two characters such as backslash and n, but the actual output contains one newline character.

## 7. Expression values and assignments

An expression computes a value. Assignment changes an existing object. Comparison computes true or false as an integer result.

```
int x = 5;
int y = x + 2;    /* y is 7 */

x = y * 3;        /* x is now 21 */

if (x == 21) {    /* comparison, not assignment */
    printf("match\n");
}
```

Use = for assignment and == for equality comparison.

## Practice Questions

Use this broken program for question 2:

```
#include <stdio.h>

int main(void) {
    int x = 4
    double y = 2.5
    printf("%d %f\n", x, y)
    return 0;
}
```

- 1 Write a complete C program that prints Ready followed by a newline and returns success.
- 2 Find and fix the missing semicolons in the broken program above.
- 3 For `int n = 7; double x = 2.5;`, choose the correct `printf` format specifiers.
- 4 Explain why a local variable declared inside a function cannot safely be returned by address.
- 5 Write one output statement that prints a tab between two numbers and a newline at the end.
- 6 Explain the difference between = and == in an if condition.
- 7 Explain why a missing semicolon can make the compiler report an error on the following line instead of the line where the semicolon is missing.
- 8 State what `return 0;` from `main` normally means.

# Mark Scheme

Use this after attempting the questions. Strong answers should connect syntax to what the compiler sees.

- 1 Expected: include `stdio.h`, define `int main(void)`, call `printf("Ready\n")`, then return `0`;
- 2 The fixed lines are `int x = 4;`, `double y = 2.5;`, and `printf("%d %f\n", x, y);`.
- 3 Use `%d` for `int` and `%f` for `double`.
- 4 The local automatic variable stops existing when the function returns, so the returned address would point at storage that is no longer valid.
- 5 Expected shape: `printf("%d\t%d\n", a, b);`.
- 6 `=` assigns a new value; `==` compares two values.
- 7 A missing semicolon leaves the previous statement unfinished, so the next line is often where the compiler first sees tokens that no longer fit the unfinished statement.
- 8 It normally reports successful program termination to the operating system.

## Common Mistakes

- Forgetting the semicolon after declarations and function calls.
- Using the wrong `printf` specifier for the argument type.
- Confusing assignment with equality comparison.
- Forgetting that local automatic variables do not live forever.
- Writing a newline as `/n` instead of `\n`.